

Loreweaver

Sistema especialista baseado em Knowledge Graph e LLMs
para a lore de Hollow Knight

João Victor Oliveira Santos · Kevin Gabriel Morais Mangueira
Luiz Henrique Santos da Graça · Pedro Henrique Paiva Souza
Victor Gabriel da Silva Menezes

O que vamos mostrar

01

O problema

Fragmentação de contexto na lore de Hollow Knight

02

A solução

Knowledge Graph + LLM + GraphRAG

03

Modelagem

Esquema fechado do grafo (Trilha B)

04

Pipeline

Da raspagem da wiki à carga no Memgraph

05

Resultados

Volumetria, gráficos e demonstração ao vivo

06

Limitações e próximos passos

O que aprendemos auditando o grafo

A lore de Hollow Knight está fragmentada

Fatos sobre um mesmo personagem, item ou local aparecem espalhados em dezenas de artigos distintos, com terminologia inconsistente e referências implícitas. Um caso extremo de fragmentação de contexto.

- Centenas de páginas na Hollow Knight Wiki (PT)
- Lore extensa, não linear e cheia de referências cruzadas
- Busca por palavra-chave não conecta fatos entre artigos



Três elementos combinados



Interface web de chat expõe o fluxo completo para consultas em linguagem natural, ao vivo.

Trilha B — Grafo de Propriedades

Escolhida por ser a mais adequada a um domínio denso em relacionamentos heterogêneos entre personagens, locais e itens, e por permitir integração direta com GraphRAG.

PIPELINE DE AQUISIÇÃO

- Raspagem via API MediaWiki (mwclient)
- Limpeza com mwparserfromhell
- Templates, navboxes e links removidos
- Conversão para Markdown

hollowknight.fandom.com/pt

365

artigos da wiki processados

→ 1 documento Markdown por artigo, servindo de corpus para a extração de triplas.

Um esquema fechado, para evitar poluição semântica

9

tipos de
nó (labels)

16

tipos de
relação

item · localizacao · npc · inimigo
habilidade · chefe · vendedor · grupo · conceito

contem · derrota · usa · localizado_em · afeta
requer · executa_habilidade · leva_a · vende
dropa · libera · cria · eh_inimigo_de
eh_relatoado_por · eh_membro_de · protege

REGRA-CHAVE DO PROMPT DE EXTRAÇÃO

Tipo mais específico sempre vence:

um vendedor de item é vendedor, não npc; a derrota de um chefe que libera uma passagem é libera, não derrota.

Proibido inventar tipos:

o modelo descarta a tripla em vez de forçá-la em um tipo incorreto.

Um pipeline de quatro etapas independentes

Python 3.12 · Poetry · cada etapa consome a saída da anterior em JSON



Por que trocamos o LLM local pelo LLM em nuvem

VERSÃO INICIAL

Ollama local

Mistral-7B-Instruct, quantizado em 4 bits

- Execução 100% offline
- Roda em GPU de 6GB VRAM
- Menor qualidade e consistência no JSON estruturado em lotes grandes

VERSÃO ATUAL

Google Gemini

gemini-3.5-flash-lite, via API em nuvem

- Maior qualidade e consistência na extração de JSON
- Depende de conectividade e de limite de taxa (14 req/min)
- Rate limiting com janela deslizante + retry/backoff em erros 429

Volumetria do grafo

1.631

tripas brutas extraídas

1.282

tripas refinadas e válidas

585

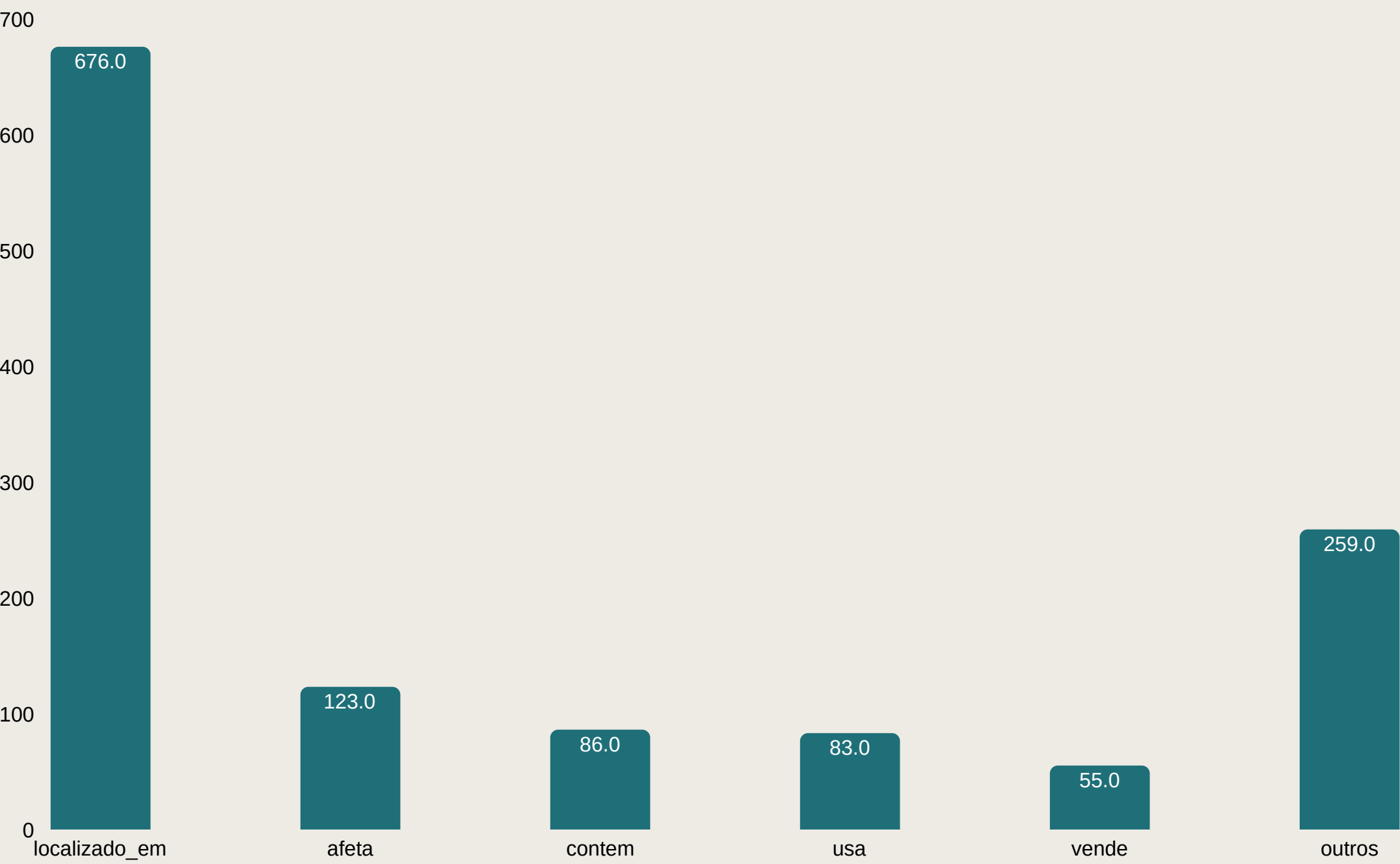
entidades únicas

21%

redução por deduplicação

1.631 tripas brutas → 1.282 refinadas, majoritariamente por deduplicação entre chunks sobrepostos e descarte de evidência vazia.

localizado_em domina o grafo

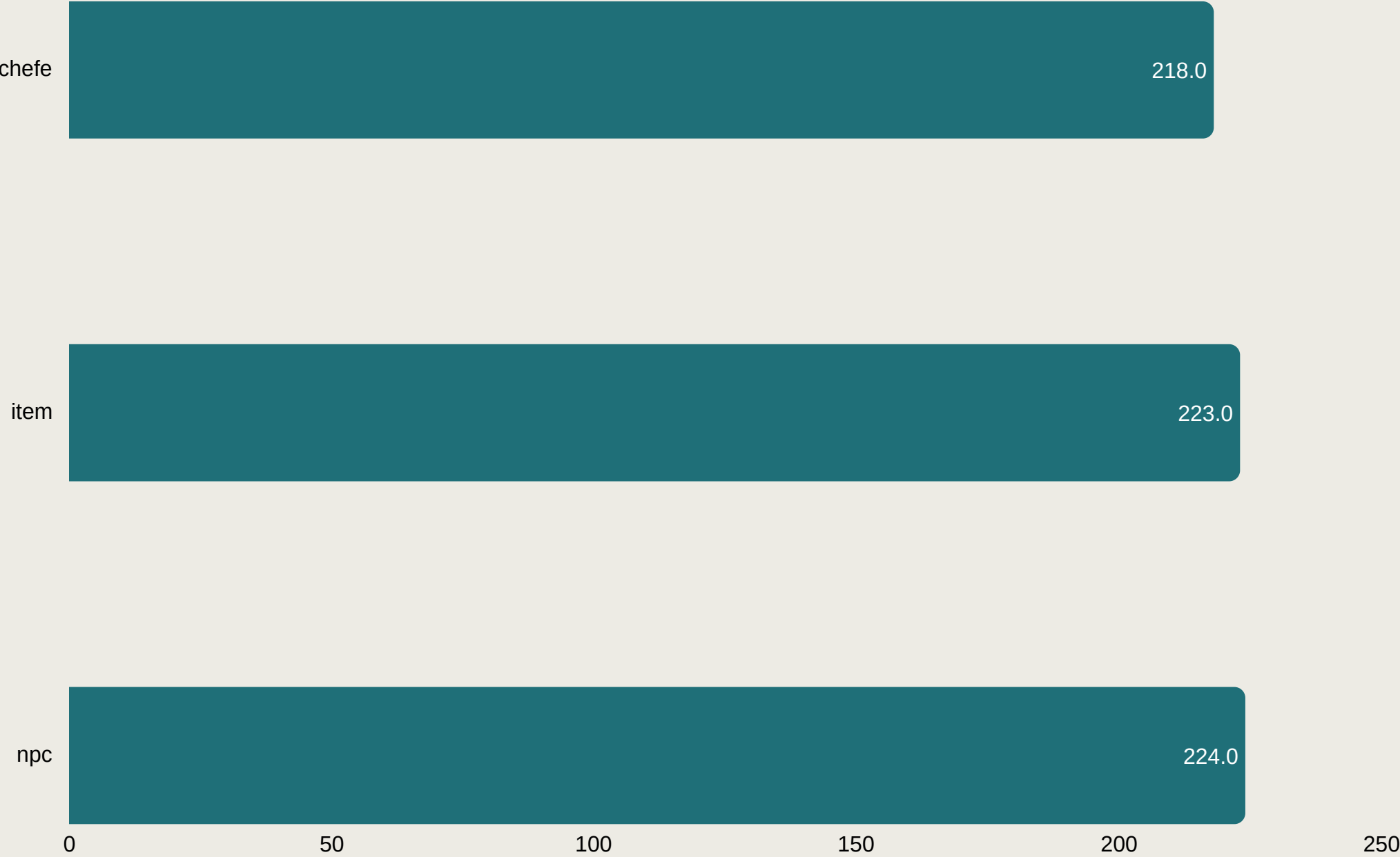


676

arestas do tipo localizado_em
53% de todas as 1.282 triplas do grafo.

*Reflete a densidade de relações espaciais no domínio:
mapas, salas e regiões conectadas entre si.*

NPCs, itens e chefes lideram como origem de triplas



POR QUE ISSO FAZ SENTIDO

Npc, item e chefe concentram a maior parte das interações narrativas do jogo (vender, derrotar, dropar, executar habilidade).

O grafo possui 585 entidades únicas no total, cobrindo 10 tipos canônicos de nó.

Pergunta em linguagem natural → resposta ancorada no grafo

"Onde o vendedor Sly pode ser encontrado e o que ele vende?"

CYPHER GERADO

```
MATCH (v:VENDEDOR) -[r:VENDE]->(i:ITEM)
WHERE toLower(v.nome) CONTAINS "sly"
RETURN v.nome, i.nome
```

RESPOSTA DO SISTEMA

Sly vende itens como a Chave Elegante e a Carapaça Robusta, ambos adquiridos em sua loja em Dirtmouth.

O FLUXO POR TRÁS DA RESPOSTA

1. Usuário faz a pergunta na interface de chat
2. Módulo text-to-Cypher traduz para Cypher
3. Consulta roda no Memgraph e recupera fatos
4. GraphRAG injeta os fatos em um 2º prompt
5. LLM responde apenas com base nesses dados

Três limitações identificadas

1

Perda de atenção em contextos longos

O modelo ocasionalmente ignora relações sutis em chunks extensos, mesmo dentro do limite de 3.000 tokens.

2

Trade-off no tamanho de lote

Lotes maiores reduzem chamadas ao LLM, mas aumentam falsos positivos na resolução de entidades.

3

Inconsistência de esquema (0,3% das arestas)

4 de 1.282 triplas usam relações fora da lista canônica e são silenciosamente reclassificadas como afeta pelo fallback da Etapa 4.

CONCLUSÃO

IA simbólica + IA generativa, com fatos rastreáveis

O Loreweaver combina a precisão do grafo de propriedades com a flexibilidade de um LLM como interface restringindo respostas aos fatos efetivamente recuperados do grafo, com rastreabilidade via a propriedade evidencia.

PRÓXIMOS PASSOS

Unificar esquema

Uma única fonte de verdade entre sinônimos e relações canônicas.

Busca híbrida

Embeddings vetoriais nos nós, combinando recuperação vetorial + Cypher.

Pré-filtro por similaridade

Similaridade de string antes do agrupamento por LLM, reduzindo lotes.



Obrigado!

Perguntas?